How can I design a reproducible methodology for extracting cleaning aggregating and visualizing commit history from the numpy/numpy GitHub repository?

```
numpy-commit-analysis/
│
├── data/
│   ├── raw/
│   ├── processed/
│
├── notebooks/
│
├── src/
│   ├── extract.py
│   ├── clean.py
│   ├── aggregate.py
│   ├── visualize.py
│
├── requirements.txt
├── README.md
└── methodology.md
```

```
pandas==2.2.2
matplotlib==3.9.0
seaborn==0.13.2
PyDriller==2.6
numpy==2.0.0
```

Bash

```
git log --numstat --date=iso \
--pretty=format:'%H|%an|%ae|%ad|%s' \
```

Bash

```
from pydriller import Repository
import pandas as pd

records = []

for commit in Repository("numpy").traverse_commits():
    records.append({
        "hash": commit.hash,
```

```
        "author": commit.author.name,
        "email": commit.author.email,
        "date": commit.author_date,
        "message": commit.msg,
        "files_changed": commit.files,
        "insertions": commit.insertions,
        "deletions": commit.deletions,
    })

df = pd.DataFrame(records)
```

```
data/raw/commits_2026_05_24.csv
```

Python Run

Python Run

```python
alias_map = {
    "jdoe@users.noreply.github.com": "John Doe",
```

Python Run

```python
```

Python Run

```python
df["year"] = df["date"].dt.year
df["month"] = df["date"].dt.to_period("M")
```

Python Run

```python
def classify(msg):
    msg = msg.lower()

    if "fix" in msg:
        return "bugfix"
    elif "doc" in msg:
        return "documentation"
    elif "refactor" in msg:
        return "refactor"
    else:
        return "other"
```

Python Run

```python
contributors = (
    df.groupby("author")
      .size()
```

Python Run

```python

```

Python Run

```
Repository: numpy/numpy
Snapshot date: 2026-05-24
HEAD commit: <hash>
```

Bash

```
python src/extract.py
python src/clean.py
python src/aggregate.py
```

Bash

```
GitHub Repo
    ↓
Git Clone
    ↓
Extraction Layer
    ↓
Raw Immutable Dataset
    ↓
Cleaning Pipeline
    ↓
Processed Analytical Dataset
    ↓
Aggregation
    ↓
Visualization + Statistics
```

```
    ↓
Report / Paper
```

How can I design a reproducible methodology for extracting cleaning aggregating and visualizing commit history from the numpy/numpy GitHub repository?

```
numpy-history-analysis/
│
├── data/
│   ├── raw/
│   ├── cleaned/
│   ├── aggregated/
│
├── notebooks/
│
```

```
├── src/
│   ├── extract.py
│   ├── clean.py
│   ├── aggregate.py
│   ├── visualize.py
│
├── outputs/
│   ├── figures/
│   ├── tables/
│
├── requirements.txt
├── README.md
└── methodology.md
```

```
Repository: numpy/numpy
Snapshot date: 2026-05-24
HEAD commit: abc123...
```

Bash

Bash

```bash
from pydriller import Repository
import pandas as pd

records = []

for commit in Repository("./numpy").traverse_commits():
    records.append({
        "hash": commit.hash,
        "author": commit.author.name,
        "email": commit.author.email,
        "date": commit.author_date,
        "message": commit.msg,
        "files_changed": commit.files,
        "insertions": commit.insertions,
        "deletions": commit.deletions,
        "modified_files": len(commit.modified_files)
    })

df = pd.DataFrame(records)
```

```
git log --numstat --date=iso \
```

Bash

```
data/raw/commits_raw_2026_05_24.csv
```

Python Run

Python Run

```
aliases = {
    "jane@users.noreply.github.com": "Jane Doe"
```

Python Run

```python
df = df[
    ~df["author"].str.contains("bot", case=False, na=False)
```

Python Run

```python
df["year"] = df["date"].dt.year
df["month"] = df["date"].dt.to_period("M")
df["weekday"] = df["date"].dt.day_name()
df["hour"] = df["date"].dt.hour
```

Python Run

```
def classify_commit(msg):
    msg = msg.lower()

    if "fix" in msg:
        return "bugfix"
    elif "doc" in msg:
        return "documentation"
    elif "refactor" in msg:
        return "refactor"
    elif "test" in msg:
        return "testing"
    else:
        return "other"
```

```
monthly_commits = (
```

Python Run

```
)
```

```
top_authors = (
    df.groupby("author")
      .size()
      .sort_values(ascending=False)
```

Python Run

Python Run

Bash

```
python src/extract.py
python src/clean.py
python src/aggregate.py
```

Bash

```
GitHub Repository
        ↓
Git Clone
        ↓
Commit Extraction
        ↓
Immutable Raw Dataset
        ↓
Cleaning + Normalisation
        ↓
Feature Engineering
        ↓
Aggregation
        ↓
Visualisation + Statistics
        ↓
Research Outputs
```

explain the code line by line and tell me which parts I should verify after running it?

```python
from pydriller import Repository
import pandas as pd

records = []

for commit in Repository("./numpy").traverse_commits():
    records.append({
        "hash": commit.hash,
        "author": commit.author.name,
        "email": commit.author.email,
        "date": commit.author_date,
        "message": commit.msg,
        "files_changed": commit.files,
        "insertions": commit.insertions,
        "deletions": commit.deletions,
        "modified_files": len(commit.modified_files)
    })

df = pd.DataFrame(records)
```

Python Run

`Repository`

`git log`

Python Run

`pd`

Python Run

```
[
  {commit1 data},
  {commit2 data},
  {commit3 data}
```

Python Run

Python Run

`Repository("./numpy")`

`./numpy`

`.git`

```
project/
│
├── analysis.py
```

```
├── numpy/
│   └── .git/
```

`.traverse_commits()`

`Commit`

```
commit1
commit2
commit3
```

Python Run

Python Run

`records`

Python Run

```
a1b2c3d4...
```

Python Run

Python Run

Python Run

```
Guido van Rossum
```

`John Doe`

`john`

`J. Doe`

```
Python
Run
```

```
Python
Run
```


```
2024-03-15 12:44:10+00:00
```

```
df["date"].min()
```

Python Run

```

```

Python Run

```
Fix memory leak in array handling
```


Python
Run


Python
Run


4


Python
Run


Python
Run


Python
Run


Python
Run


Python
Run

`commit.modified_files`

`len()`

`commit.files`

`commit.files`
`modified_files`

Python Run

```
False
```

Python Run


Python
Run


Python
Run


Python
Run

```
df['hash'].duplicated().sum()
```

Python Run

Python Run

Python Run

Python Run

Python Run


Python
Run


df["date"].min()
Python
Run


Python
Run


Python
Run

i want to try using matplotlib.pyplot instead

`matplotlib.pyplot`

Python Run

`date`
`author`
`insertions`
`deletions`

Python Run

`plt`

Python Run

Python Run

```
2025-01
2025-02
2025-03
```

Python Run

```
2024-01    120
2024-02    144
2024-03    98
```

Python Run

```
plt.figure(figsize=(12, 6))

plt.plot(monthly_commits.index,
         monthly_commits.values)

plt.title("NumPy Monthly Commit Activity")
plt.xlabel("Date")
plt.ylabel("Number of Commits")

plt.show()
```

Python Run

`figsize=(12,6)`

```
plt.plot(monthly_commits.index
```

Python Run

Python Run

Python Run


Python
Run


Python
Run


Python
Run


Python
Run

Python Run

```
2025-03
2024-01
2025-02
```

Python Run

Python Run

```
monthly_commits.index.min()
```

Python Run

Python Run

Python Run

```
plt.figure(figsize=(12,6))

plt.plot(monthly_commits.index,
         monthly_commits.values)

plt.title("NumPy Monthly Commit Activity")
plt.xlabel("Date")
plt.ylabel("Commits")

plt.xticks(rotation=45)

plt.grid(True)

plt.tight_layout()

plt.show()
```

```
top_authors = (
    df["author"]
    .value_counts()
    .head(10)
)
```

Python Run

```
plt.figure(figsize=(10,6))

plt.bar(top_authors.index,
```

```
        top_authors.values)

plt.title("Top 10 Contributors")
plt.xlabel("Author")
plt.ylabel("Commits")

plt.xticks(rotation=45)

plt.tight_layout()

plt.show()
```

```
jdoe
John Doe
john-doe
```

```
dependabot
github-actions
```

Python Run

```
plt.figure(figsize=(10,6))

plt.hist(df["churn"], bins=50)

plt.title("Distribution of Commit Churn")
plt.xlabel("Lines Changed")
plt.ylabel("Frequency")
```

Python Run

Python Run

`dpi=300`

```
# Extract
# Clean
```

```
# Aggregate

monthly_commits = (
    df.groupby(df["date"].dt.to_period("M"))
      .size()
)

monthly_commits.index = (
    monthly_commits.index.to_timestamp()
)

# Visualize

plt.figure(figsize=(12,6))

plt.plot(monthly_commits.index,
         monthly_commits.values)

plt.title("Monthly Commit Activity")
plt.xlabel("Date")
plt.ylabel("Commits")

plt.xticks(rotation=45)

plt.grid(True)

plt.tight_layout()

plt.show()
```

how can I create simple Matplotlib charts for monthly commit count and monthly code churn and after that save them as image files for the report?

`df`

`date`

`insertions`

`deletions`

```python
monthly_commits = (
    df.groupby(df["date"].dt.to_period("M"))
      .size()
```

Python Run

```python
monthly_commits.index = (
    monthly_commits.index.to_timestamp()
```

Python Run

```python
plt.figure(figsize=(12,6))

plt.plot(
    monthly_commits.index,
    monthly_commits.values
)

plt.title("Monthly Commit Count")
plt.xlabel("Month")
plt.ylabel("Number of Commits")

plt.xticks(rotation=45)

plt.grid(True)

plt.tight_layout()
```

Python Run

`plt.show()`

```
plt.figure(figsize=(12,6))

plt.plot(
    monthly_commits.index,
    monthly_commits.values
)

plt.title("Monthly Commit Count")
plt.xlabel("Month")
plt.ylabel("Number of Commits")

plt.xticks(rotation=45)

plt.grid(True)

plt.tight_layout()

plt.savefig(
```

```
    "monthly_commit_count.png",
    dpi=300,
    bbox_inches="tight"
)

plt.show()
```

Python Run

Python Run

Python Run

```
insertions + deletions
```

```
df["churn"] = (
    df["insertions"] +
    df["deletions"]
```

Python Run

```
monthly_churn = (
    df.groupby(df["date"].dt.to_period("M"))["churn"]
      .sum()
```

Python Run

```
monthly_churn.index = (
    monthly_churn.index.to_timestamp()
```

Python Run

```
plt.figure(figsize=(12,6))

plt.plot(
    monthly_churn.index,
    monthly_churn.values
)

plt.title("Monthly Code Churn")
plt.xlabel("Month")
plt.ylabel("Lines Changed")

plt.xticks(rotation=45)

plt.grid(True)

plt.tight_layout()

plt.savefig(
    "monthly_code_churn.png",
    dpi=300,
    bbox_inches="tight"
)

plt.show()
```

Python Run

```
monthly_commits.head()
monthly_churn.head()
```

Python Run

```
monthly_commits.index
```

Python

```python
monthly_churn.sort_values(ascending=False).head(10)
```

```
monthly_commit_count.png
monthly_code_churn.png
```

Python

```python
plt.plot(
    monthly_commits.index,
    monthly_commits.values,
    linewidth=2
)
```

Python

Run

```python
import pandas as pd
import matplotlib.pyplot as plt

df["date"] = pd.to_datetime(df["date"])

# Commit counts
monthly_commits = (
    df.groupby(df["date"].dt.to_period("M"))
      .size()
)

monthly_commits.index = (
    monthly_commits.index.to_timestamp()
)

plt.figure(figsize=(12,6))

plt.plot(
    monthly_commits.index,
    monthly_commits.values
)

plt.title("Monthly Commit Count")
plt.xlabel("Month")
plt.ylabel("Commits")

plt.xticks(rotation=45)

plt.grid(True)

plt.tight_layout()

plt.savefig(
    "monthly_commit_count.png",
    dpi=300,
    bbox_inches="tight"
)

plt.show()
```

```
# Code churn
df["churn"] = (
    df["insertions"] +
    df["deletions"]
)

monthly_churn = (
    df.groupby(df["date"].dt.to_period("M"))["churn"]
      .sum()
)

monthly_churn.index = (
    monthly_churn.index.to_timestamp()
)

plt.figure(figsize=(12,6))

plt.plot(
    monthly_churn.index,
    monthly_churn.values
)

plt.title("Monthly Code Churn")
plt.xlabel("Month")
plt.ylabel("Lines Changed")

plt.xticks(rotation=45)

plt.grid(True)

plt.tight_layout()

plt.savefig(
    "monthly_code_churn.png",
    dpi=300,
    bbox_inches="tight"
)

plt.show()
```